

in the subtree rooted at x . Show how to update the f attributes in $O(1)$ time after a rotation. Modify your argument slightly to apply it to the $size$ attributes in order-statistic trees.

17.3 Interval trees

This section shows how to augment red-black trees to support operations on dynamic sets of intervals. In this section, we'll assume that intervals are closed. Extending the results to open and half-open intervals is conceptually straightforward. (See page 1157 for definitions of closed, open, and half-open intervals.)

Intervals are convenient for representing events that each occupy a continuous period of time. For example, you could query a database of time intervals to find out which events occurred during a given interval. The data structure in this section provides an efficient means for maintaining such an interval database.

A simple way to represent an interval $[t_1, t_2]$ is as an object i with attributes $i.low = t_1$ (the **low endpoint**) and $i.high = t_2$ (the **high endpoint**). We say that intervals i and i' **overlap** if $i \cap i' \neq \emptyset$, that is, if $i.low \leq i'.high$ and $i'.low \leq i.high$.

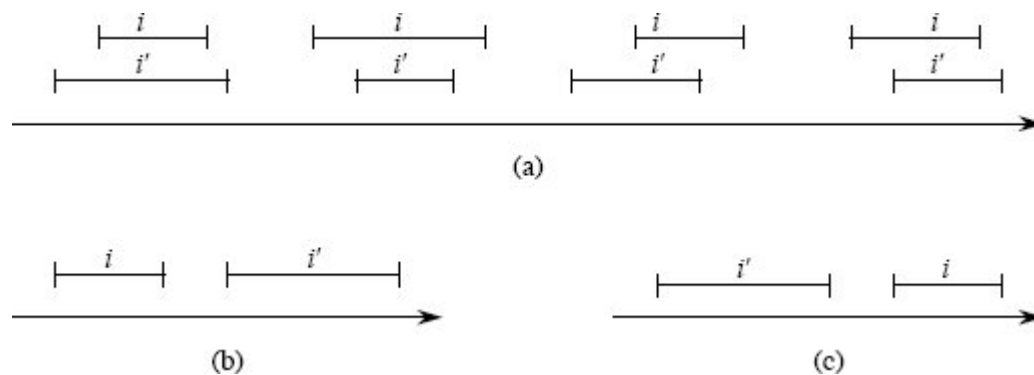


Figure 17.3 The interval trichotomy for two closed intervals i and i' . (a) If i and i' overlap, there are four situations, and in each, $i.low \leq i'.high$ and $i'.low \leq i.high$. (b) The intervals do not overlap, and $i.high < i'.low$. (c) The intervals do not overlap, and $i'.high < i.low$.

As Figure 17.3 shows, any two intervals i and i' satisfy the **interval trichotomy**, that is, exactly one of the following three properties holds:

- a. i and i' overlap,
- b. i is to the left of i' (i.e., $i.high < i'.low$),
- c. i is to the right of i' (i.e., $i'.high < i.low$).

An *interval tree* is a red-black tree that maintains a dynamic set of elements, with each element x containing an interval $x.int$. Interval trees support the following operations:

INTERVAL-INSERT(T, x) adds the element x , whose *int* attribute is assumed to contain an interval, to the interval tree T .

INTERVAL-DELETE(T, x) removes the element x from the interval tree T .

INTERVAL-SEARCH(T, i) returns a pointer to an element x in the interval tree T such that $x.int$ overlaps interval i , or a pointer to the sentinel $T.nil$ if no such element belongs to the set.

Figure 17.4 shows how an interval tree represents a set of intervals. The four-step method from Section 17.2 will guide our design of an interval tree and the operations that run on it.

Step 1: Underlying data structure

A red-black tree serves as the underlying data structure. Each node x contains an interval $x.int$. The key of x is the low endpoint, $x.int.low$, of the interval. Thus, an inorder tree walk of the data structure lists the intervals in sorted order by low endpoint.

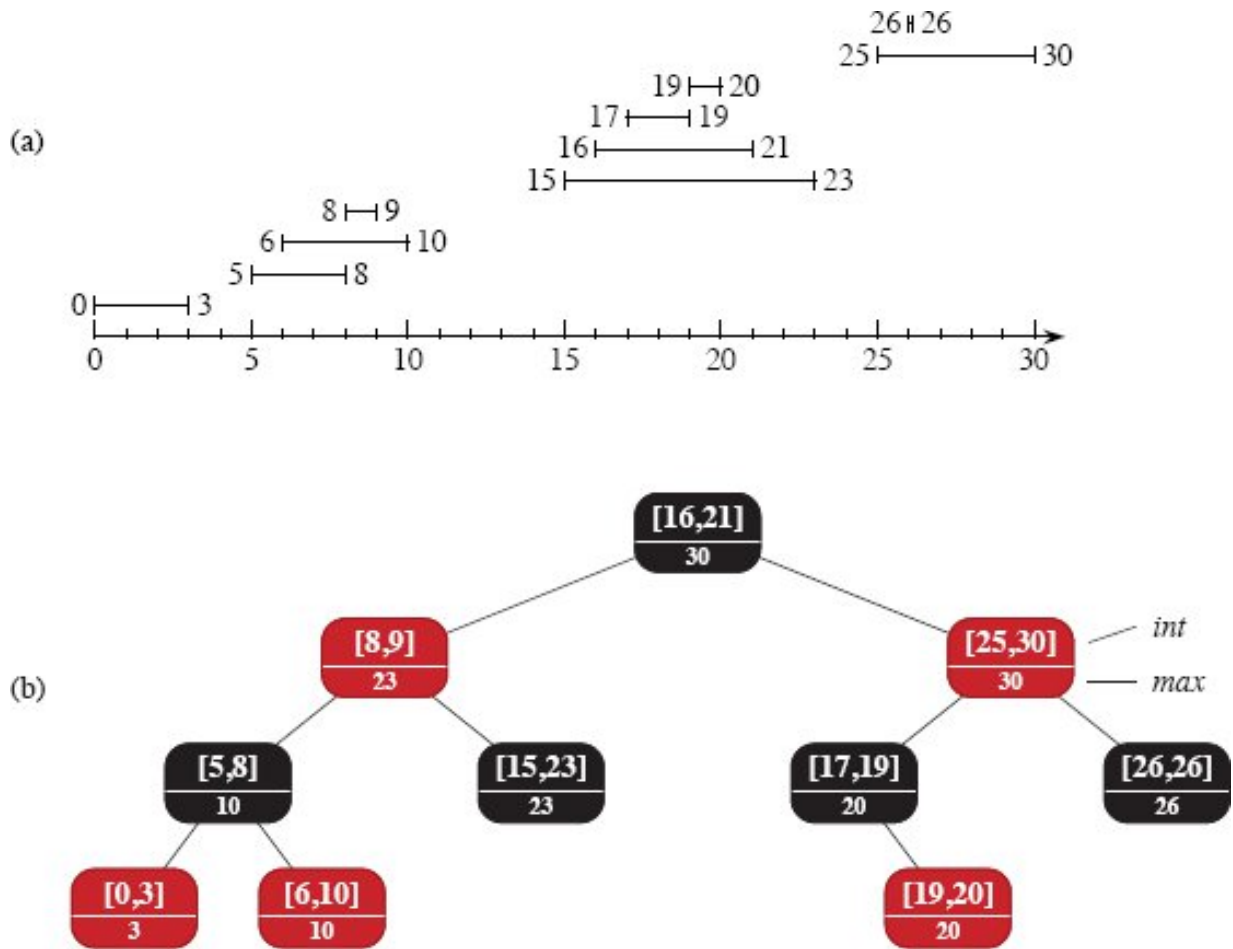


Figure 17.4 An interval tree. (a) A set of 10 intervals, shown sorted bottom to top by left endpoint. (b) The interval tree that represents them. Each node x contains an interval, shown above the dashed line, and the maximum value of any interval endpoint in the subtree rooted at x , shown below the dashed line. An inorder tree walk of the tree lists the nodes in sorted order by left endpoint.

Step 2: Additional information

In addition to the intervals themselves, each node x contains a value $x.max$, which is the maximum value of any interval endpoint stored in the subtree rooted at x .

Step 3: Maintaining the information

We must verify that insertion and deletion take $O(\lg n)$ time on an interval tree of n nodes. It is simple enough to determine $x.max$ in $O(1)$

time, given interval $x.int$ and the *max* values of node x 's children:

$x.max = \max \{x.int.high, x.left.max, x.right.max\}.$

Thus, by Theorem 17.1, insertion and deletion run in $O(\lg n)$ time. In fact, you can use either Exercise 17.2-3 or 17.3-1 to show how to update all the *max* attributes that change after a rotation in just $O(1)$ time.

Step 4: Developing new operations

The only new operation is INTERVAL-SEARCH(T, i), which finds a node in tree T whose interval overlaps interval i . If there is no interval in the tree that overlaps i , the procedure returns a pointer to the sentinel $T.nil$.

INTERVAL-SEARCH(T, i)

```
1  $x = T.root$ 
2 while  $x \neq T.nil$  and  $i$  does not overlap  $x.int$ 
3   if  $x.left \neq T.nil$  and  $x.left.max \geq i.low$ 
4      $x = x.left$  // overlap in left subtree or no overlap in right
                   subtree
5   else    $x = x.right$  // no overlap in left subtree
6 return  $x$ 
```

The search for an interval that overlaps i starts at the root of the tree and proceeds downward. It terminates when either it finds an overlapping interval or it reaches the sentinel $T.nil$. Since each iteration of the basic loop takes $O(1)$ time, and since the height of an n -node red-black tree is $O(\lg n)$, the INTERVAL-SEARCH procedure takes $O(\lg n)$ time.

Before we see why INTERVAL-SEARCH is correct, let's examine how it works on the interval tree in Figure 17.4. Let's look for an interval that overlaps the interval $i = [22, 25]$. Begin with x as the root, which contains $[16, 21]$ and does not overlap i . Since $x.left.max = 23$ is greater than $i.low = 22$, the loop continues with x as the left child of the root—the node containing $[8, 9]$, which also does not overlap i . This

time, $x.\text{left.max} = 10$ is less than $i.\text{low} = 22$, and so the loop continues with the right child of x as the new x . Because the interval $[15, 23]$ stored in this node overlaps i , the procedure returns this node.

Now let's try an unsuccessful search, for an interval that overlaps $i = [11, 14]$ in the interval tree of Figure 17.4. Again, begin with x as the root. Since the root's interval $[16, 21]$ does not overlap i , and since $x.\text{left.max} = 23$ is greater than $i.\text{low} = 11$, go left to the node containing $[8, 9]$. Interval $[8, 9]$ does not overlap i , and $x.\text{left.max} = 10$ is less than $i.\text{low} = 11$, and so the search goes right. (No interval in the left subtree overlaps i .) Interval $[15, 23]$ does not overlap i , and its left child is $T.\text{nil}$, so again the search goes right, the loop terminates, and INTERVAL-SEARCH returns the sentinel $T.\text{nil}$.

To see why INTERVAL-SEARCH is correct, we must understand why it suffices to examine a single path from the root. The basic idea is that at any node x , if $x.\text{int}$ does not overlap i , the search always proceeds in a safe direction: the search will definitely find an overlapping interval if the tree contains one. The following theorem states this property more precisely.

Theorem 17.2

Any execution of INTERVAL-SEARCH(T, i) either returns a node whose interval overlaps i , or it returns $T.\text{nil}$ and the tree T contains no node whose interval overlaps i .

Proof The **while** loop of lines 2–5 terminates when either $x = T.\text{nil}$ or i overlaps $x.\text{int}$. In the latter case, it is certainly correct to return x . Therefore, we focus on the former case, in which the **while** loop terminates because $x = T.\text{nil}$, which is the node that INTERVAL-SEARCH returns.

We'll prove that if the procedure returns $T.\text{nil}$, then it did not miss any intervals in T that overlap i . The idea is to show that whether the search goes left in line 4 or right in line 5, it always heads toward a node containing an interval overlapping i , if any such interval exists. In particular, we'll prove that

1. If the search goes left in line 4, then the left subtree of node x contains an interval that overlaps i or the right subtree of x contains no interval that overlaps i . Therefore, even if x 's left subtree contains no interval that overlaps i but the search goes left, it does not make a mistake, because x 's right subtree does not contain an interval overlapping i , either.
2. If the search goes right in line 5, then the left subtree of x contains no interval that overlaps i . Thus, if the search goes right, it does not make a mistake.

For both cases, we rely on the interval trichotomy. Let's start with the case where the search goes right, whose proof is simpler. By the tests in line 3, we know that $x.left = T.nil$ or $x.left.max < i.low$. If $x.left = T.nil$, then x 's left subtree contains no interval that overlaps i , since it contains no intervals at all. Now suppose that $x.left \neq T.nil$, so that we must have $x.left.max < i.low$. Consider any interval i' in x 's left subtree. Because $x.left.max$ is the maximum endpoint in x 's left subtree, we have $i'.high \leq x.left.max$. Thus, as Figure 17.5(a) shows,

$$\begin{aligned} i'.high &\leq x.left.max \\ &< i.low. \end{aligned}$$

By the interval trichotomy, therefore, intervals i and i' do not overlap, and so x 's left subtree contains no interval that overlaps i .

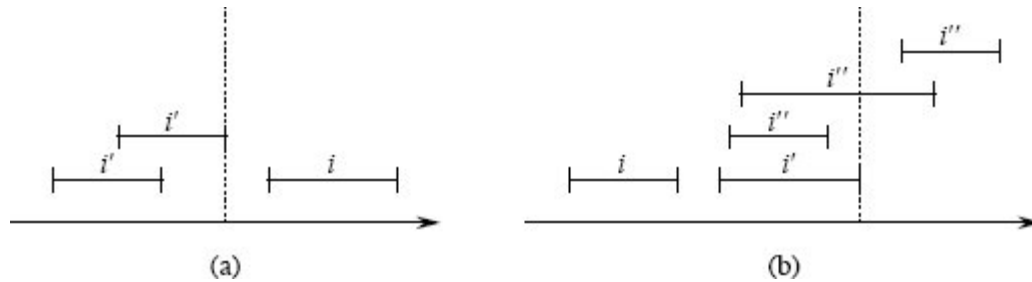


Figure 17.5 Intervals in the proof of Theorem 17.2. The value of $x.left.max$ is shown in each case as a dashed line. **(a)** The search goes right. No interval i' in x 's left subtree can overlap i . **(b)** The search goes left. The left subtree of x contains an interval that overlaps i (situation not shown), or x 's left subtree contains an interval i' such that $i'.high = x.left.max$. Since i does not overlap i' , neither does it overlap any interval i'' in x 's right subtree, since $i'.low \leq i''.low$.

Now we examine the case in which the search goes left. If the left subtree of node x contains an interval that overlaps i , we're done, so let's assume that no node in x 's left subtree overlaps i . We need to show that in this case, no node in x 's right subtree overlaps i , so that going left will not miss any overlaps in x 's right subtree. By the tests in line 3, the left subtree of x is not empty and $x.\text{left.max} \geq i.\text{low}$. By the definition of the *max* attribute, x 's left subtree contains some interval i' such that

$$\begin{aligned} i'.\text{high} &= x.\text{left.max} \\ &\geq i.\text{low}, \end{aligned}$$

as illustrated in Figure 17.5(b). Since i' is in x 's left subtree, it does not overlap i , and since $i'.\text{high} \geq i.\text{low}$, the interval trichotomy tells us that $i.\text{high} < i'.\text{low}$. Now we bring in the property that interval trees are keyed on the low endpoints of intervals. Because i' is in x 's left subtree, we have $i'.\text{low} \leq x.\text{int.low}$. Now consider any interval i'' in x 's right subtree, so that $x.\text{int.low} \leq i''.\text{low}$. Putting inequalities together, we get

$$\begin{aligned} i.\text{high} &< i'.\text{low} \\ &\leq x.\text{int.low} \\ &\leq i''.\text{low}. \end{aligned}$$

Because $i.\text{high} < i''.\text{low}$, the interval trichotomy tells us that i and i'' do not overlap. Since we chose i'' as any interval in x 's right subtree, no node in x 's right subtree overlaps i . ■

Thus, the INTERVAL-SEARCH procedure works correctly.

Exercises

17.3-1

Write pseudocode for LEFT-ROTATE that operates on nodes in an interval tree and updates all the *max* attributes that change in $O(1)$ time.

17.3-2

Describe an efficient algorithm that, given an interval i , returns an interval overlapping i that has the minimum low endpoint, or $T.nil$ if no such interval exists.

17.3-3

Given an interval tree T and an interval i , describe how to list all intervals in T that overlap i in $O(\min \{n, k \lg n\})$ time, where k is the number of intervals in the output list. (*Hint*: One simple method makes several queries, modifying the tree between queries. A slightly more complicated method does not modify the tree.)

17.3-4

Suggest modifications to the interval-tree procedures to support the new operation INTERVAL-SEARCH-EXACTLY(T, i), where T is an interval tree and i is an interval. The operation should return a pointer to a node x in T such that $x.int.low = i.low$ and $x.int.high = i.high$, or $T.nil$ if T contains no such node. All operations, including INTERVAL-SEARCH-EXACTLY, should run in $O(\lg n)$ time on an n -node interval tree.

17.3-5

Show how to maintain a dynamic set Q of numbers that supports the operation MIN-GAP, which gives the absolute value of the difference of the two closest numbers in Q . For example, if we have $Q = \{1, 5, 9, 15, 18, 22\}$, then MIN-GAP(Q) returns 3, since 15 and 18 are the two closest numbers in Q . Make the operations INSERT, DELETE, SEARCH, and MIN-GAP as efficient as possible, and analyze their running times.

★ 17.3-6

VLSI databases commonly represent an integrated circuit as a list of rectangles. Assume that each rectangle is rectilinearly oriented (sides parallel to the x - and y -axes), so that each rectangle is represented by four values: its minimum and maximum x - and y -coordinates. Give an $O(n \lg n)$ -time algorithm to decide whether a set of n rectangles so represented contains two rectangles that overlap. Your algorithm need not report all intersecting pairs, but it must report that an overlap exists

if one rectangle entirely covers another, even if the boundary lines do not intersect. (*Hint*: Move a “sweep” line across the set of rectangles.)

Problems

17-1 *Point of maximum overlap*

You wish to keep track of a *point of maximum overlap* in a set of intervals—a point with the largest number of intervals in the set that overlap it.

- a. Show that there is always a point of maximum overlap that is an endpoint of one of the intervals.
- b. Design a data structure that efficiently supports the operations INTERVAL-INSERT, INTERVAL-DELETE, and FIND-POM, which returns a point of maximum overlap. (*Hint*: Keep a red-black tree of all the endpoints. Associate a value of +1 with each left endpoint, and associate a value of −1 with each right endpoint. Augment each node of the tree with some extra information to maintain the point of maximum overlap.)

17-2 *Josephus permutation*

We define the *Josephus problem* as follows. A group of n people form a circle, and we are given a positive integer $m \leq n$. Beginning with a designated first person, proceed around the circle, removing every m th person. After each person is removed, counting continues around the circle that remains. This process continues until nobody remains in the circle. The order in which the people are removed from the circle defines the *(n, m) -Josephus permutation* of the integers $1, 2, \dots, n$. For example, the $(7, 3)$ -Josephus permutation is $\langle 3, 6, 2, 7, 5, 1, 4 \rangle$.

- a. Suppose that m is a constant. Describe an $O(n)$ -time algorithm that, given an integer n , outputs the (n, m) -Josephus permutation.
- b. Suppose that m is not necessarily a constant. Describe an $O(n \lg n)$ -time algorithm that, given integers n and m , outputs the (n, m) -